

BÁO CÁO MÃ NGUỒN - LAB03

****Sinh viên thực hiện:** Lê Viết Đức**

****MSSV:** 23DTHA4 / 0533**

****Môn học:** Thực hành Bảo mật thông tin nâng cao**

Đường dẫn file: `lab03\api\_server.py`

```
import sys
import os
from flask import Flask, request, jsonify

# Ensure the lab03 folder is in path for modules import
sys.path.append(os.path.dirname(os.path.abspath(__file__)))

from crypto.rsa_cipher import generate_rsa_keys, rsa_encrypt, rsa_decrypt
from crypto.ecc_signature import generate_ecc_keys, ecc_sign, ecc_verify

app = Flask(__name__)

# Global variables to store keys in memory
rsa_private_key = None
rsa_public_key = None

ecc_private_key = None
ecc_public_key = None

@app.route('/', methods=['GET'])
def index():
    return jsonify({
        "message": "Lab03 RSA ECC API is running"
    })

@app.route('/rsa/generate-key', methods=['POST'])
def rsa_generate_key():
    global rsa_private_key, rsa_public_key
    try:
        rsa_private_key, rsa_public_key = generate_rsa_keys()
        return jsonify({
            "message": "RSA key generated successfully",
            "public_key": rsa_public_key,
            "private_key": rsa_private_key
        })
    except Exception as e:
        return jsonify({"message": str(e)}), 500
```

```

    })
except Exception as e:
    return jsonify({"error": f"Không thể tạo khóa RSA: {str(e)}"}), 400

@app.route('/rsa/encrypt', methods=['POST'])
def rsa_encrypt_endpoint():
    global rsa_public_key
    if not rsa_public_key:
        return jsonify({"error": "Khóa công khai RSA chưa được tạo. Vui lòng gọi /rsa/generate-key trước."}), 400

    data = request.get_json(silent=True)
    if not data or 'plaintext' not in data:
        return jsonify({"error": "Thiếu tham số bắt buộc 'plaintext' trong JSON body."}), 400

    plaintext = data['plaintext']
    try:
        ciphertext = rsa_encrypt(plaintext, rsa_public_key)
        return jsonify({
            "plaintext": plaintext,
            "ciphertext": ciphertext
        })
    except Exception as e:
        return jsonify({"error": str(e)}), 400

@app.route('/rsa/decrypt', methods=['POST'])
def rsa_decrypt_endpoint():
    global rsa_private_key
    if not rsa_private_key:
        return jsonify({"error": "Khóa bí mật RSA chưa được tạo. Vui lòng gọi /rsa/generate-key trước."}), 400

    data = request.get_json(silent=True)
    if not data or 'ciphertext' not in data:
        return jsonify({"error": "Thiếu tham số bắt buộc 'ciphertext' trong JSON body."}), 400

    ciphertext = data['ciphertext']
    try:
        decrypted_text = rsa_decrypt(ciphertext, rsa_private_key)
        return jsonify({
            "decrypted_text": decrypted_text
        })
    except Exception as e:
        return jsonify({"error": f"Giải mã thất bại: {str(e)}"}), 400

@app.route('/ecc/generate-key', methods=['POST'])
def ecc_generate_key_endpoint():
    global ecc_private_key, ecc_public_key
    try:
        ecc_private_key, ecc_public_key = generate_ecc_keys()
        return jsonify({

```

```

        "message": "ECC key generated successfully",
        "public_key": ecc_public_key,
        "private_key": ecc_private_key
    })
except Exception as e:
    return jsonify({"error": f"Không thể tạo khóa ECC: {str(e)}"}), 400

@app.route('/ecc/sign', methods=['POST'])
def ecc_sign_endpoint():
    global ecc_private_key
    if not ecc_private_key:
        return jsonify({"error": "Khóa bí mật ECC chưa được tạo. Vui lòng gọi /ecc/generate-key trước."}), 400

    data = request.get_json(silent=True)
    if not data or 'message' not in data:
        return jsonify({"error": "Thiếu tham số bắt buộc 'message' trong JSON body."}), 400

    message = data['message']
    try:
        signature = ecc_sign(message, ecc_private_key)
        return jsonify({
            "message": message,
            "signature": signature
        })
    except Exception as e:
        return jsonify({"error": str(e)}), 400

@app.route('/ecc/verify', methods=['POST'])
def ecc_verify_endpoint():
    global ecc_public_key
    if not ecc_public_key:
        return jsonify({"error": "Khóa công khai ECC chưa được tạo. Vui lòng gọi /ecc/generate-key trước."}), 400

    data = request.get_json(silent=True)
    if not data:
        return jsonify({"error": "Thiếu dữ liệu JSON body."}), 400

    if 'message' not in data:
        return jsonify({"error": "Thiếu tham số bắt buộc 'message' trong JSON body."}), 400
    if 'signature' not in data:
        return jsonify({"error": "Thiếu tham số bắt buộc 'signature' trong JSON body."}), 400

    message = data['message']
    signature = data['signature']

    try:
        is_valid = ecc_verify(message, signature, ecc_public_key)
        if is_valid:
            return jsonify({

```

```

        "valid": True,
        "message": "Signature is valid"
    })
else:
    return jsonify({
        "valid": False,
        "message": "Signature is invalid"
    })
except Exception as e:
    return jsonify({"error": f"Xác minh thất bại: {str(e)}"}), 400

if __name__ == '__main__':
    app.run(host='127.0.0.1', port=5000, debug=True)

```

Đường dẫn file: `lab03\main.py`

```

import sys
import os
from PyQt5.QtWidgets import QApplication, QWidget
from PyQt5 import uic

# Ensure the lab03 folder is in path for modules import
sys.path.append(os.path.dirname(os.path.abspath(__file__)))

from ui.rsa_window import RSAWindow
from ui.ecc_window import ECCWindow

class MainApplication(QWidget):
    def __init__(self):
        super().__init__()

        # Load the UI file dynamically
        ui_path = os.path.join(os.path.dirname(os.path.abspath(__file__)), 'ui', 'main.ui')
        uic.loadUi(ui_path, self)

        self.rsa_window = None
        self.ecc_window = None

        self.init_connections()

    def init_connections(self):
        self.btn_rsa.clicked.connect(self.open_rsa_window)
        self.btn_ecc.clicked.connect(self.open_ecc_window)

    def open_rsa_window(self):
        if not self.rsa_window:

```

```

        self.rsa_window = RSAWindow()
        self.rsa_window.show()
        self.rsa_window.raise_()
        self.rsa_window.activateWindow()

def open_ecc_window(self):
    if not self.ecc_window:
        self.ecc_window = ECCWindow()
        self.ecc_window.show()
        self.ecc_window.raise_()
        self.ecc_window.activateWindow()

def main():
    app = QApplication(sys.argv)

    # Premium Dark Theme Stylesheet
    dark_stylesheet = """
QWidget {
    background-color: #1e1e24;
    color: #e4e4e6;
    font-family: 'Segoe UI', Helvetica, Arial, sans-serif;
}

QTextEdit, QLineEdit {
    background-color: #2b2b36;
    border: 1px solid #3f3f52;
    border-radius: 6px;
    padding: 8px;
    color: #ffffff;
    selection-background-color: #5865f2;
}

QTextEdit:focus, QLineEdit:focus {
    border: 1px solid #5865f2;
}

QPushButton {
    background-color: #5865f2;
    border: none;
    border-radius: 6px;
    color: white;
    font-weight: bold;
    padding: 10px 18px;
}

QPushButton:hover {
    background-color: #4752c4;
}

QPushButton:pressed {
    background-color: #3c45a5;

```

```

    }

    QLabel {
        font-weight: bold;
        color: #b9bbbe;
    }

    QMessageBox {
        background-color: #2f3136;
        color: #ffffff;
    }

    QMessageBox QLabel {
        color: #ffffff;
        font-weight: normal;
    }

    QMessageBox QPushButton {
        background-color: #4f545c;
        min-width: 80px;
    }

    QMessageBox QPushButton:hover {
        background-color: #686d73;
    }
"""

app.setStyleSheet(dark_stylesheet)

window = MainApplication()
window.show()
sys.exit(app.exec_())

if __name__ == "__main__":
    main()

```

Đường dẫn file: `lab03\requirements.txt`

```

PyQt5
pycryptodome
Flask

```

Đường dẫn file: `lab03\crypto\\_init\_.py`

```
# Crypto package
```

Đường dẫn file: `lab03\crypto\ecc\_signature.py`

```
from Crypto.PublicKey import ECC
from Crypto.Signature import DSS
from Crypto.Hash import SHA256
import base64

def generate_ecc_keys():
    """
    Generate ECC key pair using P-256 curve.
    Returns:
        tuple: (private_key_pem, public_key_pem) as strings.
    """
    key = ECC.generate(curve='P-256')
    private_key_pem = key.export_key(format='PEM')
    public_key_pem = key.public_key().export_key(format='PEM')
    return private_key_pem, public_key_pem

def ecc_sign(message: str, private_key_pem: str) -> str:
    """
    Sign a message using the ECC private key and SHA256 + DSS/ECDSA.
    Args:
        message (str): Message to sign.
        private_key_pem (str): ECC private key in PEM format.
    Returns:
        str: Signature in Base64 encoded format.
    """
    try:
        key = ECC.import_key(private_key_pem)
        h = SHA256.new(message.encode('utf-8'))
        signer = DSS.new(key, 'fips-186-3')
        signature = signer.sign(h)
        return base64.b64encode(signature).decode('utf-8')
    except Exception as e:
        raise ValueError(f"Ký thông điệp thất bại: {str(e)}")

def ecc_verify(message: str, signature_b64: str, public_key_pem: str) -> bool:
    """
    Verify a signature using the ECC public key and SHA256 + DSS/ECDSA.
    Args:
        message (str): Original or modified message.
```

```

signature_b64 (str): Base64 encoded signature.
public_key_pem (str): ECC public key in PEM format.
Returns:
    bool: True if signature is valid, False otherwise.
"""
try:
    key = ECC.import_key(public_key_pem)
    h = SHA256.new(message.encode('utf-8'))
    verifier = DSS.new(key, 'fips-186-3')
    signature = base64.b64decode(signature_b64.encode('utf-8'))
    verifier.verify(h, signature)
    return True
except (ValueError, TypeError, Exception):
    return False

```

Đường dẫn file: `lab03\crypto\rsa\_cipher.py`

```

from Crypto.PublicKey import RSA
from Crypto.Cipher import PKCS1_OAEP
import base64

def generate_rsa_keys():
    """
    Generate RSA 2048-bit key pair.
    Returns:
        tuple: (private_key_pem, public_key_pem) as strings.
    """
    key = RSA.generate(2048)
    private_key_pem = key.export_key().decode('utf-8')
    public_key_pem = key.publickey().export_key().decode('utf-8')
    return private_key_pem, public_key_pem

def rsa_encrypt(plaintext: str, public_key_pem: str) -> str:
    """
    Encrypt plaintext using the RSA public key (PKCS1_OAEP).
    Args:
        plaintext (str): The text to encrypt.
        public_key_pem (str): RSA public key in PEM format.
    Returns:
        str: Ciphertext in Base64 encoded format.
    """
    try:
        recipient_key = RSA.import_key(public_key_pem)
        cipher_rsa = PKCS1_OAEP.new(recipient_key)
        ciphertext = cipher_rsa.encrypt(plaintext.encode('utf-8'))
        return base64.b64encode(ciphertext).decode('utf-8')
    
```



```

except Exception as e:
    raise ValueError(f"Mã hóa thất bại: {str(e)}")

def rsa_decrypt(ciphertext_b64: str, private_key_pem: str) -> str:
    """
    Decrypt Base64 ciphertext using the RSA private key (PKCS1_OAEP).
    Args:
        ciphertext_b64 (str): Base64 encoded ciphertext.
        private_key_pem (str): RSA private key in PEM format.
    Returns:
        str: Decrypted plaintext.
    """
    try:
        private_key = RSA.import_key(private_key_pem)
        cipher_rsa = PKCS1_OAEP.new(private_key)
        ciphertext = base64.b64decode(ciphertext_b64.encode('utf-8'))
        decrypted = cipher_rsa.decrypt(ciphertext)
        return decrypted.decode('utf-8')
    except Exception as e:
        raise ValueError(f"Giải mã thất bại. Vui lòng kiểm tra lại khóa hoặc bản mã: {str(e)}")

```

Đường dẫn file: `lab03\ui\\_\_init\_\_.py`

```
# UI package
```

Đường dẫn file: `lab03\ui\ecc.ui`

```

<?xml version="1.0" encoding="UTF-8"?>
<ui version="4.0">
  <class>ECCWidget</class>
  <widget class="QWidget" name="ECCWidget">
    <property name="geometry">
      <rect>
        <x>0</x>
        <y>0</y>
        <width>800</width>
        <height>600</height>
      </rect>
    </property>
    <property name="windowTitle">
      <string>ECC Digital Signature</string>
    </property>

```

```

<layout class="QVBoxLayout" name="mainVerticalLayout">
  <property name="spacing">
    <number>15</number>
  </property>
  <property name="leftMargin">
    <number>20</number>
  </property>
  <property name="topMargin">
    <number>20</number>
  </property>
  <property name="rightMargin">
    <number>20</number>
  </property>
  <property name="bottomMargin">
    <number>20</number>
  </property>
  <item>
    <widget class="QLabel" name="title_label">
      <property name="text">
        <string>ECC P-256 Digital Signature (ECDSA)</string>
      </property>
      <property name="styleSheet">
        <string>font-size: 18px; font-weight: bold; color: #5865f2; margin-bottom: 10px;</string>
      </property>
    </widget>
  </item>
  <item>
    <layout class="QGridLayout" name="keysGridLayout">
      <property name="spacing">
        <number>10</number>
      </property>
      <item row="0" column="0">
        <widget class="QLabel" name="pubLabel">
          <property name="text">
            <string>Public Key (PEM):</string>
          </property>
        </widget>
      </item>
      <item row="1" column="0">
        <widget class="QTextEdit" name="pub_key_txt">
          <property name="readOnly">
            <bool>true</bool>
          </property>
          <property name="placeholderText">
            <string>Khóa công khai (Public Key) sẽ hiển thị ở đây...</string>
          </property>
          <property name="maximumHeight">
            <number>150</number>
          </property>
        </widget>
      </item>
    </layout>
  </item>

```

```

<item row="0" column="1">
  <widget class="QLabel" name="privLabel">
    <property name="text">
      <string>Private Key (PEM):</string>
    </property>
  </widget>
</item>
<item row="1" column="1">
  <widget class="QTextEdit" name="priv_key_txt">
    <property name="readOnly">
      <bool>true</bool>
    </property>
    <property name="placeholderText">
      <string>Khóa bí mật (Private Key) sẽ hiển thị ở đây...</string>
    </property>
    <property name="maximumHeight">
      <number>150</number>
    </property>
  </widget>
</item>
</layout>
</item>
<item>
  <widget class="QPushButton" name="btn_gen_key">
    <property name="text">
      <string>Generate ECC Key</string>
    </property>
    <property name="cursor">
      <cursorShape>PointingHandCursor</cursorShape>
    </property>
    <property name="styleSheet">
      <string>padding: 10px; font-size: 14px;</string>
    </property>
  </widget>
</item>
<item>
  <widget class="QLabel" name="msgLabel">
    <property name="text">
      <string>Message (Thông điệp):</string>
    </property>
  </widget>
</item>
<item>
  <widget class="QTextEdit" name="message_txt">
    <property name="placeholderText">
      <string>Nhập nội dung cần ký...</string>
    </property>
    <property name="maximumHeight">
      <number>100</number>
    </property>
  </widget>

```

```

</item>
<item>
<layout class="QHBoxLayout" name="actionsHorizontalLayout">
  <property name="spacing">
    <number>20</number>
  </property>
  <item>
    <widget class="QPushButton" name="btn_sign">
      <property name="text">
        <string>Sign Message (Ký chữ ký)</string>
      </property>
      <property name="cursor">
        <cursorShape>PointingHandCursor</cursorShape>
      </property>
    </widget>
  </item>
  <item>
    <widget class="QPushButton" name="btn_verify">
      <property name="text">
        <string>Verify Signature (Xác minh)</string>
      </property>
      <property name="cursor">
        <cursorShape>PointingHandCursor</cursorShape>
      </property>
    </widget>
  </item>
</layout>
</item>
<item>
<widget class="QLabel" name="sigLabel">
  <property name="text">
    <string>Signature (Base64) (Chữ ký):</string>
  </property>
</widget>
</item>
<item>
<widget class="QTextEdit" name="signature_txt">
  <property name="placeholderText">
    <string>Chữ ký số dạng Base64...</string>
  </property>
  <property name="maximumHeight">
    <number>80</number>
  </property>
</widget>
</item>
<item>
<layout class="QHBoxLayout" name="statusHorizontalLayout">
  <item>
    <widget class="QLabel" name="statusLabel">
      <property name="text">
        <string>Kết quả xác minh (Verification Result):</string>

```

```

        </property>
    </widget>
</item>
<item>
    <widget class="QLineEdit" name="status_val">
        <property name="readOnly">
            <bool>true</bool>
        </property>
        <property name="placeholderText">
            <string>Chưa thực hiện xác minh</string>
        </property>
        <property name="styleSheet">
            <string>font-weight: bold; font-size: 14px; text-align: center; padding: 5px;</string>
        </property>
    </widget>
</item>
</layout>
</item>
<item>
    <widget class="QPushButton" name="btn_back">
        <property name="text">
            <string>Quay lại Menu</string>
        </property>
        <property name="styleSheet">
            <string>background-color: #4f545c; font-weight: normal; margin-top: 10px;</string>
        </property>
    </widget>
</item>
</layout>
</widget>
<resources/>
<connections/>
</ui>

```

Đường dẫn file: `lab03\ui\ecc\_window.py`

```

import os
from PyQt5.QtWidgets import QWidget, QMessageBox
from PyQt5 import uic
from crypto.ecc_signature import generate_ecc_keys, ecc_sign, ecc_verify

class ECCWindow(QWidget):
    def __init__(self, parent=None):
        super().__init__(parent)

        # Load the UI file dynamically

```

```

ui_path = os.path.join(os.path.dirname(os.path.abspath(__file__)), 'ecc.ui')
uic.loadUi(ui_path, self)

# Local keys storage
self.private_key = None
self.public_key = None

self.init_connections()

def init_connections(self):
    # Connect buttons defined in ecc.ui to handler methods
    self.btn_gen_key.clicked.connect(self.handle_generate_key)
    self.btn_sign.clicked.connect(self.handle_sign)
    self.btn_verify.clicked.connect(self.handle_verify)
    self.btn_back.clicked.connect(self.close)

def handle_generate_key(self):
    try:
        self.private_key, self.public_key = generate_ecc_keys()
        self.pub_key_txt.setPlainText(self.public_key)
        self.priv_key_txt.setPlainText(self.private_key)
        QMessageBox.information(self, "Thành công", "Sinh cặp khóa ECC P-256 thành công!")
    except Exception as e:
        QMessageBox.critical(self, "Lỗi", f"Không thể sinh cặp khóa: {str(e)}")

def handle_sign(self):
    if not self.private_key:
        QMessageBox.warning(self, "Lỗi", "Vui lòng sinh khóa ECC trước khi thực hiện ký!")
        return

    message = self.message_txt.toPlainText()
    if not message:
        QMessageBox.warning(self, "Thông báo", "Vui lòng nhập Message cần ký!")
        return

    try:
        signature = ecc_sign(message, self.private_key)
        self.signature_txt.setPlainText(signature)
        self.status_val.clear()
        self.status_val.setPlaceholderText("Chưa thực hiện xác minh")
        self.status_val.setStyleSheet("font-weight: bold; font-size: 14px; padding: 5px;
color: #b9bbbe; background-color: #2b2b36;")
    except Exception as e:
        QMessageBox.critical(self, "Lỗi ký", str(e))

def handle_verify(self):
    if not self.public_key:
        QMessageBox.warning(self, "Lỗi", "Vui lòng sinh khóa ECC trước khi xác minh chữ
ký!")

    return

```

```

message = self.message_txt.toPlainText()
signature = self.signature_txt.toPlainText().strip()

if not signature:
    QMessageBox.warning(self, "Thông báo", "Vui lòng có chữ ký (Signature) để xác
minh!")
    return

is_valid = ecc_verify(message, signature, self.public_key)

if is_valid:
    self.status_val.setText("Chữ ký HỢP LỆ (Signature is Valid)")
    self.status_val.setStyleSheet("font-weight: bold; font-size: 14px; padding: 5px;
color: #43b581; background-color: #2b2b36; border: 1px solid #43b581;")
    QMessageBox.information(self, "Kết quả xác minh", "Chữ ký hợp lệ!")
else:
    self.status_val.setText("Chữ ký KHÔNG HỢP LỆ (Signature is Invalid)")
    self.status_val.setStyleSheet("font-weight: bold; font-size: 14px; padding: 5px;
color: #f04747; background-color: #2b2b36; border: 1px solid #f04747;")
    QMessageBox.warning(self, "Kết quả xác minh", "Chữ ký KHÔNG hợp lệ! Bản tin có thể
đã bị sửa đổi hoặc chữ ký không đúng.")

```

Đường dẫn file: `lab03\ui\main.ui`

```

<?xml version="1.0" encoding="UTF-8"?>
<ui version="4.0">
<class>MainWindow</class>
<widget class="QWidget" name="MainWindow">
<property name="geometry">
<rect>
<x>0</x>
<y>0</y>
<width>500</width>
<height>380</height>
</rect>
</property>
<property name="windowTitle">
<string>Lab 03 - RSA & ECC Security Application</string>
</property>
<layout class="QVBoxLayout" name="verticalLayout">
<property name="spacing">
<number>20</number>
</property>
<property name="leftMargin">
<number>30</number>
</property>

```

```

<property name="topMargin">
  <number>30</number>
</property>
<property name="rightMargin">
  <number>30</number>
</property>
<property name="bottomMargin">
  <number>30</number>
</property>
<item>
  <widget class="QLabel" name="title_label">
    <property name="text">
      <string>Lab 03 - RSA & ECC Security Application</string>
    </property>
    <property name="alignment">
      <set>Qt::AlignCenter</set>
    </property>
    <property name="styleSheet">
      <string>font-size: 20px; font-weight: bold; color: #5865f2; margin-bottom: 5px;</string>
    </property>
  </widget>
</item>
<item>
  <widget class="QLabel" name="author_label">
    <property name="text">
      <string>Sinh viên: Lê Việt Đức - MSSV: 0533 - Lớp: 23DTHA4</string>
    </property>
    <property name="alignment">
      <set>Qt::AlignCenter</set>
    </property>
    <property name="styleSheet">
      <string>font-size: 13px; color: #b9bbbe;</string>
    </property>
  </widget>
</item>
<item>
  <widget class="QLabel" name="course_label">
    <property name="text">
      <string>Môn: Thực hành Bảo mật thông tin nâng cao</string>
    </property>
    <property name="alignment">
      <set>Qt::AlignCenter</set>
    </property>
    <property name="styleSheet">
      <string>font-size: 13px; color: #b9bbbe; font-style: italic;</string>
    </property>
  </widget>
</item>
<item>
  <widget class="QPushButton" name="btn_rsa">
    <property name="text">

```



```

    <string>1. RSA Encryption / Decryption</string>
  </property>
  <property name="cursor">
    <cursorShape>PointingHandCursor</cursorShape>
  </property>
  <property name="styleSheet">
    <string>padding: 15px; font-size: 14px; text-align: left; padding-left: 25px;</string>
  </property>
</widget>
</item>
<item>
  <widget class="QPushButton" name="btn_ecc">
    <property name="text">
      <string>2. ECC Digital Signature</string>
    </property>
    <property name="cursor">
      <cursorShape>PointingHandCursor</cursorShape>
    </property>
    <property name="styleSheet">
      <string>padding: 15px; font-size: 14px; text-align: left; padding-left: 25px;</string>
    </property>
  </widget>
</item>
<item>
  <widget class="QLabel" name="footer_label">
    <property name="text">
      <string>Chọn một chức năng để bắt đầu</string>
    </property>
    <property name="alignment">
      <set>Qt::AlignCenter</set>
    </property>
    <property name="styleSheet">
      <string>color: #72767d; font-size: 11px;</string>
    </property>
  </widget>
</item>
</layout>
</widget>
<resources/>
<connections/>
</ui>

```

Đường dẫn file: `lab03\ui\rsa.ui`

```

<?xml version="1.0" encoding="UTF-8"?>
<ui version="4.0">

```

```

<class>RSAWidget</class>
<widget class="QWidget" name="RSAWidget">
  <property name="geometry">
    <rect>
      <x>0</x>
      <y>0</y>
      <width>800</width>
      <height>600</height>
    </rect>
  </property>
  <property name="windowTitle">
    <string>RSA Encryption / Decryption</string>
  </property>
  <layout class="QVBoxLayout" name="mainVerticalLayout">
    <property name="spacing">
      <number>15</number>
    </property>
    <property name="leftMargin">
      <number>20</number>
    </property>
    <property name="topMargin">
      <number>20</number>
    </property>
    <property name="rightMargin">
      <number>20</number>
    </property>
    <property name="bottomMargin">
      <number>20</number>
    </property>
    <item>
      <widget class="QLabel" name="title_label">
        <property name="text">
          <string>RSA 2048-bit Encryption & Decryption</string>
        </property>
        <property name="styleSheet">
          <string>font-size: 18px; font-weight: bold; color: #5865f2; margin-bottom: 10px;</string>
        </property>
      </widget>
    </item>
    <item>
      <layout class="QGridLayout" name="keysGridLayout">
        <property name="spacing">
          <number>10</number>
        </property>
        <item row="0" column="0">
          <widget class="QLabel" name="pubLabel">
            <property name="text">
              <string>Public Key (PEM):</string>
            </property>
          </widget>
        </item>
      </layout>
    </item>
  </layout>
</widget>

```

```

<item row="1" column="0">
  <widget class="QTextEdit" name="pub_key_txt">
    <property name="readOnly">
      <bool>true</bool>
    </property>
    <property name="placeholderText">
      <string>Khóa công khai (Public Key) sẽ hiển thị ở đây...</string>
    </property>
    <property name="maximumHeight">
      <number>150</number>
    </property>
  </widget>
</item>
<item row="0" column="1">
  <widget class="QLabel" name="privLabel">
    <property name="text">
      <string>Private Key (PEM):</string>
    </property>
  </widget>
</item>
<item row="1" column="1">
  <widget class="QTextEdit" name="priv_key_txt">
    <property name="readOnly">
      <bool>true</bool>
    </property>
    <property name="placeholderText">
      <string>Khóa bí mật (Private Key) sẽ hiển thị ở đây...</string>
    </property>
    <property name="maximumHeight">
      <number>150</number>
    </property>
  </widget>
</item>
</layout>
</item>
<item>
  <widget class="QPushButton" name="btn_gen_key">
    <property name="text">
      <string>Generate RSA Key</string>
    </property>
    <property name="cursor">
      <cursorShape>PointingHandCursor</cursorShape>
    </property>
    <property name="styleSheet">
      <string>padding: 10px; font-size: 14px;</string>
    </property>
  </widget>
</item>
<item>
  <layout class="QHBoxLayout" name="contentHorizontalLayout">
    <property name="spacing">

```

```

<number>20</number>
</property>
<item>
<layout class="QVBoxLayout" name="leftVerticalLayout">
  <item>
    <widget class="QLabel" name="plainLabel">
      <property name="text">
        <string>Plaintext (Bản rõ):</string>
      </property>
    </widget>
  </item>
  <item>
    <widget class="QTextEdit" name="plaintext_txt">
      <property name="placeholderText">
        <string>Nhập nội dung cần mã hóa...</string>
      </property>
    </widget>
  </item>
  <item>
    <widget class="QPushButton" name="btn_encrypt">
      <property name="text">
        <string>Encrypt (Mã hóa)</string>
      </property>
      <property name="cursor">
        <cursorShape>PointingHandCursor</cursorShape>
      </property>
    </widget>
  </item>
</layout>
</item>
<item>
<layout class="QVBoxLayout" name="rightVerticalLayout">
  <item>
    <widget class="QLabel" name="cipherLabel">
      <property name="text">
        <string>Ciphertext (Base64) (Bản mã):</string>
      </property>
    </widget>
  </item>
  <item>
    <widget class="QTextEdit" name="ciphertext_txt">
      <property name="placeholderText">
        <string>Bản mã dạng Base64...</string>
      </property>
    </widget>
  </item>
  <item>
    <widget class="QPushButton" name="btn_decrypt">
      <property name="text">
        <string>Decrypt (Giải mã)</string>
      </property>

```

```

        <property name="cursor">
            <cursorShape>PointingHandCursor</cursorShape>
        </property>
    </widget>
</item>
<item>
    <widget class="QLabel" name="decryptedLabel">
        <property name="text">
            <string>Decrypted Text (Bản giải mã):</string>
        </property>
    </widget>
</item>
<item>
    <widget class="QTextEdit" name="decrypted_txt">
        <property name="readOnly">
            <bool>true</bool>
        </property>
        <property name="placeholderText">
            <string>Nội dung sau khi giải mã...</string>
        </property>
        <property name="maximumHeight">
            <number>100</number>
        </property>
    </widget>
</item>
</layout>
</item>
</layout>
</item>
<item>
    <widget class="QPushButton" name="btn_back">
        <property name="text">
            <string>Quay lại Menu</string>
        </property>
        <property name="styleSheet">
            <string>background-color: #4f545c; font-weight: normal; margin-top: 10px;</string>
        </property>
    </widget>
</item>
</layout>
</widget>
<resources/>
<connections/>
</ui>

```

Đường dẫn file: `lab03\ui\rsa\_window.py`

```

import os
from PyQt5.QtWidgets import QWidget, QMessageBox
from PyQt5 import uic
from crypto.rsa_cipher import generate_rsa_keys, rsa_encrypt, rsa_decrypt

class RSAWindow(QWidget):
    def __init__(self, parent=None):
        super().__init__(parent)

        # Load the UI file dynamically
        ui_path = os.path.join(os.path.dirname(os.path.abspath(__file__)), 'rsa.ui')
        uic.loadUi(ui_path, self)

        # Local keys storage
        self.private_key = None
        self.public_key = None

        self.init_connections()

    def init_connections(self):
        # Connect buttons defined in rsa.ui to handler methods
        self.btn_gen_key.clicked.connect(self.handle_generate_key)
        self.btn_encrypt.clicked.connect(self.handle_encrypt)
        self.btn_decrypt.clicked.connect(self.handle_decrypt)
        self.btn_back.clicked.connect(self.close)

    def handle_generate_key(self):
        try:
            self.private_key, self.public_key = generate_rsa_keys()
            self.pub_key_txt.setPlainText(self.public_key)
            self.priv_key_txt.setPlainText(self.private_key)
            QMessageBox.information(self, "Thành công", "Sinh cặp khóa RSA 2048-bit thành
công!")
        except Exception as e:
            QMessageBox.critical(self, "Lỗi", f"Không thể sinh cặp khóa: {str(e)}")

    def handle_encrypt(self):
        if not self.public_key:
            QMessageBox.warning(self, "Lỗi", "Vui lòng sinh khóa RSA trước khi thực hiện mã
hóa!")
            return

        plaintext = self.plaintext_txt.toPlainText()
        if not plaintext:
            QMessageBox.warning(self, "Thông báo", "Vui lòng nhập Plaintext cần mã hóa!")
            return

        try:
            ciphertext = rsa_encrypt(plaintext, self.public_key)
            self.ciphertext_txt.setPlainText(ciphertext)

```

```

except Exception as e:
    QMessageBox.critical(self, "Lỗi mã hóa", str(e))

def handle_decrypt(self):
    if not self.private_key:
        QMessageBox.warning(self, "Lỗi", "Vui lòng sinh khóa RSA trước khi thực hiện giải
mã!")
        return

    ciphertext = self.ciphertext_txt.toPlainText().strip()
    if not ciphertext:
        QMessageBox.warning(self, "Thông báo", "Vui lòng nhập Ciphertext cần giải mã!")
        return

    try:
        decrypted = rsa_decrypt(ciphertext, self.private_key)
        self.decrypted_txt.setPlainText(decrypted)
    except Exception as e:
        self.decrypted_txt.setPlainText("Lỗi giải mã")
        QMessageBox.warning(self, "Lỗi giải mã", "Giải mã không thành công! Bản mã đã bị
thay đổi hoặc khóa không khớp.")

```